

UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA
FACULTAD DE INGENIERIA EN SISTEMAS DE INFORMACION
CURSO: PROGRAMACIÓN SECCION C



Maycol Gerardo Cabrera Cruz 9941-24-21588
Arleth Susana Orozco Paredes 9941-24-18927

GUATEMALA, 9 DE NOVIEMBRE DE 2025

Introducción

Sabemos que en la actualidad la tecnología tiene un papel importante e influyente en nuestra vida, la usamos para facilitar y optimizar procesos de nuestro día a día, últimamente se utiliza en mucho en el trabajo moderno por ejemplo se usa en automatización de procesos, también en el área académica se utiliza demasiado por los estudiantes ya que se opta por la búsqueda de información y expansión de conocimientos, podemos decir que la tecnología ha sido fundamental para la humanidad ya que a lo largo de la historia ha contribuido con el desarrollo en diferentes ámbitos de la vida cotidiana, lo mas revolucionario que se vive hoy en día es el uso de la IA (Inteligencia Artificial) la cual ha superado expectativas y ha sorprendido su capacidad en desarrollo y aprendizaje, podemos decir que en si la IA ha reemplazado puntos fuertes de muchas áreas y asi mismo ha generado muchas oportunidades en todo sentido.

En este contexto, el presente proyecto de programación tiene como finalidad el desarrollo de un sistema de logística y envíos conectado a una base de datos, diseñado para facilitar la administración y control de las operaciones relacionadas con la recepción, almacenamiento y distribución de productos. Posteriormente, el sistema fue adaptado e implementado en un entorno **web**, permitiendo su acceso desde cualquier dispositivo con conexión a Internet y mejorando la comunicación entre los distintos actores del proceso logístico.

Proyecto de logística y envío

Este proyecto se realizó utilizando diversos sistemas en los cuales se realizó por medio de Java Script junto con Spring Boot y se realizó la conexión a base de datos Maria DB para posteriormente conectar el programa a postman para la realización de las APIs

El proyecto consiste en un sistema de logística y envíos, cuyo objetivo es gestionar de manera eficiente el registro de paquetes, el seguimiento de los envíos y la administración de clientes y repartidores. El sistema permite automatizar procesos que normalmente se realizan de forma manual, reduciendo errores y mejorando la organización.

Como se menciona anteriormente las herramientas que utilizamos para la realización del proyecto fueron:

Explica brevemente el rol de cada herramienta:

Tecnología	Uso en el Proyecto
Java con Spring Boot	Desarrollo del backend y lógica del negocio (controladores, servicios, repositorios).
MariaDB	Base de datos donde se almacenan los datos de envíos, usuarios, repartidores, etc.
DataGrip	Herramienta que utilicé para administrar y visualizar la base de datos.
JavaScript	Implementación en el frontend para la interacción del usuario con el sistema.
Postman	Utilizado para probar las APIs del backend y verificar que las rutas funcionen correctamente.

El funcionamiento en si del proyecto consta en estos principales puntos

1. El cliente (frontend) envía solicitudes (por ejemplo: *crear envío*, *consultar estado*).
2. El backend en Spring Boot recibe la solicitud, procesa la lógica y consulta la base de datos.
3. MariaDB responde con la información solicitada.
4. El backend devuelve la respuesta al frontend.

Las funcionalidades principales son:

- Registro de clientes y repartidores.
- Registro de paquetes.
- Consulta del estado del envío.
- Actualización de la ubicación/status del paquete.
- Listado y filtrado de envíos.
- Validación de datos.

También se uso Postman para verificar que cada endpoint funcionara correctamente. Probé solicitudes GET, POST, PUT y DELETE, comprobando que los datos se guardaran y se actualizaran en la base de datos.

Para entender realmente el funcionamiento del sistema es importante conocer las herramientas que utilizamos.

Spring Boot nos permitió construir la lógica del servidor y los servicios que procesan la

información.

MariaDB se encargó del almacenamiento de los datos como clientes, paquetes y estados de los envíos.

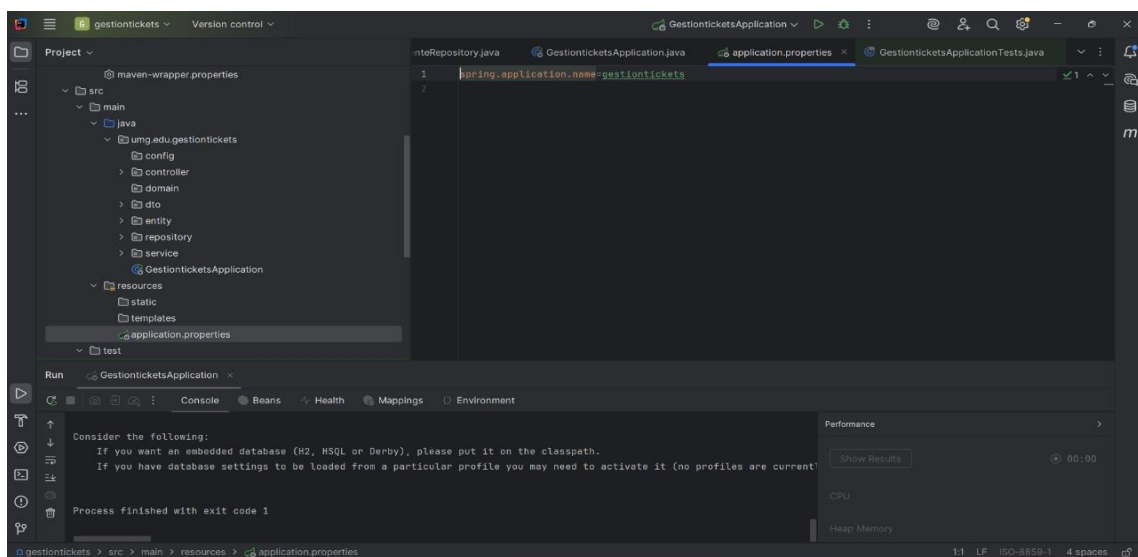
Con DataGrip administramos y verificamos la estructura de la base de datos.

JavaScript nos permitió manejar la parte visual y la interacción del usuario con el sistema.

Finalmente, Postman nos ayudó a probar y asegurar que todas las funciones del sistema respondieran correctamente.

Gracias a la integración de estas herramientas, el sistema funciona de manera eficiente, organizada y confiable. Esto logramos aprenderlo mayormente en nuestra clase de programación ya que es impresionante la cantidad y diversidad de procesos y conocimientos informáticos que nos ayudan a desarrollar lo que imaginemos, en algún lugar lei algo que decía “si puedes imaginarlo, puedes programarlo” y cada vez nos damos cuenta de lo cierto que es y el impacto que tiene sobre nuestras vidas académicas y el impacto que tendrá en nuestra vida laboral, profesional y personal, sin mas preámbulo pasamos a lo que es nuestra documentación en la cual damos a conocer el correcto funcionamiento del proyecto.

Empezamos creando nuestras clases en Java



The image shows a code editor with a file explorer on the left. The file explorer shows a project structure for 'umg.edu.logisticayenvios' with folders for 'controller', 'dto', 'entity', 'repository', and 'service'. The 'controller' folder is expanded, showing 'ClienteController', 'EntregaController', 'OrdenEnvioController', and 'TransportistaController'. The 'EntregaController' is selected, and its code is displayed in the main editor. The code includes HTML templates and JavaScript functions for deleting and saving clients. The bottom status bar shows '85 problems'.

```

2      <html lang="es">
3      </html>
4
5      <body>
6      </body>
7
8      <script>
9
10         document.addEventListener("DOMContentLoaded", () => {
11
12             async function loadClientes() { Show usages
13
14                 <button class="btn btn-danger" onClick="deleteCliente({c.id})">Eliminar</button>
15             </td>
16         </tr>
17     </tr>
18     </tbody>
19 </html>
20
21     <script>
22
23         async function saveCliente(ev) { Show usages
24
25             ev.preventDefault();
26             const id = $('#cliente-id').value;
27             const body = {
28                 nombre: $('#cliente-nombre').value,
29                 direccion: $('#cliente-direccion').value,
30                 telefono: $('#cliente-telefono').value
31             };
32             fetch('/api/clientes', {
33                 method: 'POST',
34                 headers: {
35                     'Content-Type': 'application/json'
36                 },
37                 body: JSON.stringify(body)
38             })
39                 .then(response => response.json())
40                 .then(data => {
41                     alert('Cliente guardado exitosamente');
42                     loadClientes();
43                 })
44                 .catch(error => {
45                     alert('Error al guardar el cliente');
46                 });
47         }
48     </script>
49 </script>
50 </body>
51 </html>
52
53     <script>
54         async function deleteCliente(c) { Show usages
55
56             fetch(`/api/clientes/${c.id}`, {
57                 method: 'DELETE'
58             })
59                 .then(response => response.json())
60                 .then(data => {
61                     alert('Cliente eliminado exitosamente');
62                     loadClientes();
63                 })
64                 .catch(error => {
65                     alert('Error al eliminar el cliente');
66                 });
67         }
68     </script>
69 </script>
70 </body>
71 </html>
72
73     <script>
74         async function saveCliente(ev) { Show usages
75
76             ev.preventDefault();
77             const id = $('#cliente-id').value;
78             const body = {
79                 nombre: $('#cliente-nombre').value,
80                 direccion: $('#cliente-direccion').value,
81                 telefono: $('#cliente-telefono').value
82             };
83             fetch('/api/clientes', {
84                 method: 'POST',
85                 headers: {
86                     'Content-Type': 'application/json'
87                 },
88                 body: JSON.stringify(body)
89             })
90                 .then(response => response.json())
91                 .then(data => {
92                     alert('Cliente guardado exitosamente');
93                     loadClientes();
94                 })
95                 .catch(error => {
96                     alert('Error al guardar el cliente');
97                 });
98         }
99     </script>
100 </script>
101 </body>
102 </html>

```

Problems 85 Project Errors Server-Side Analysis Vulnerable Dependencies Security Analysis New x

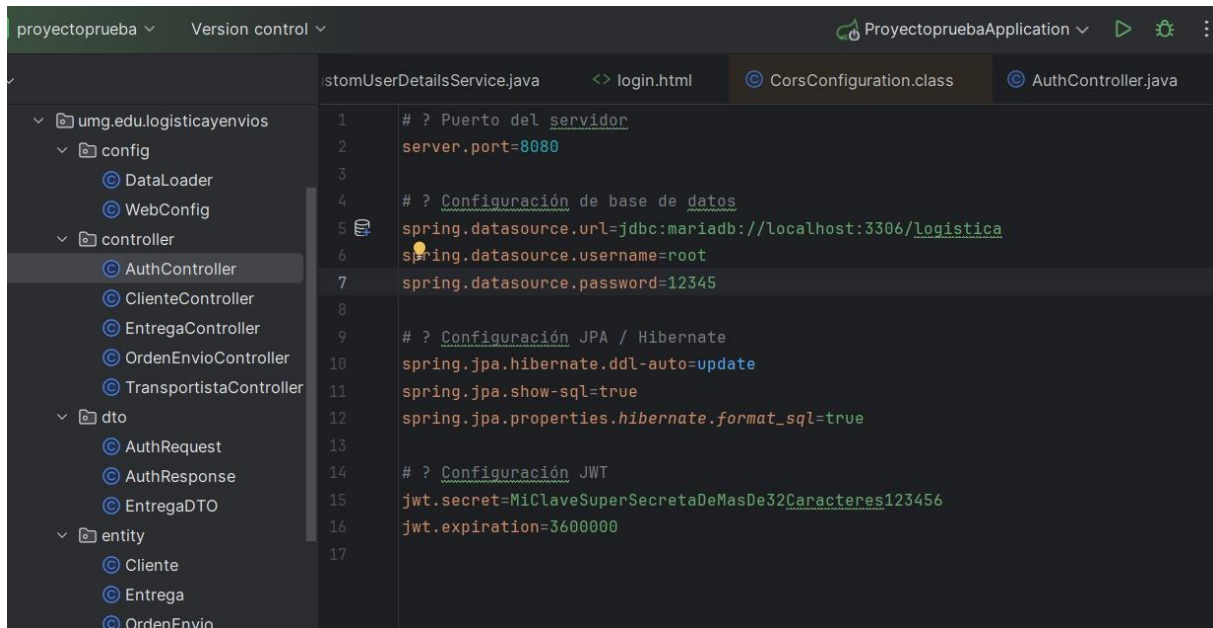
<> Proyecto Final.html C:\Users\Arieth\Downloads\JDK (2)\JDK\projectoprueba\src\main\resources\static 85 problems

- Font property font-family does not have generic default :15
- Selector hidden is never used :22
- Missing associated label :51

The screenshot displays the MySQL Workbench interface. On the left, the 'Schemas' pane shows the 'logistica' database selected, with a list of tables: 'clientes' (16.0 KiB), 'entregas' (48.0 KiB), 'ordenes_envio' (32.0 KiB), 'roles' (32.0 KiB), 'transportistas' (16.0 KiB), 'usuarios' (32.0 KiB), and 'usuario_roles' (32.0 KiB). The 'clientes' table is highlighted. On the right, the 'Table' pane shows the 'clientes' table with 3 rows. The table has 4 columns: '#', '1', '2 direccion', '3 nombre', and '4 telefono'. The data is as follows:

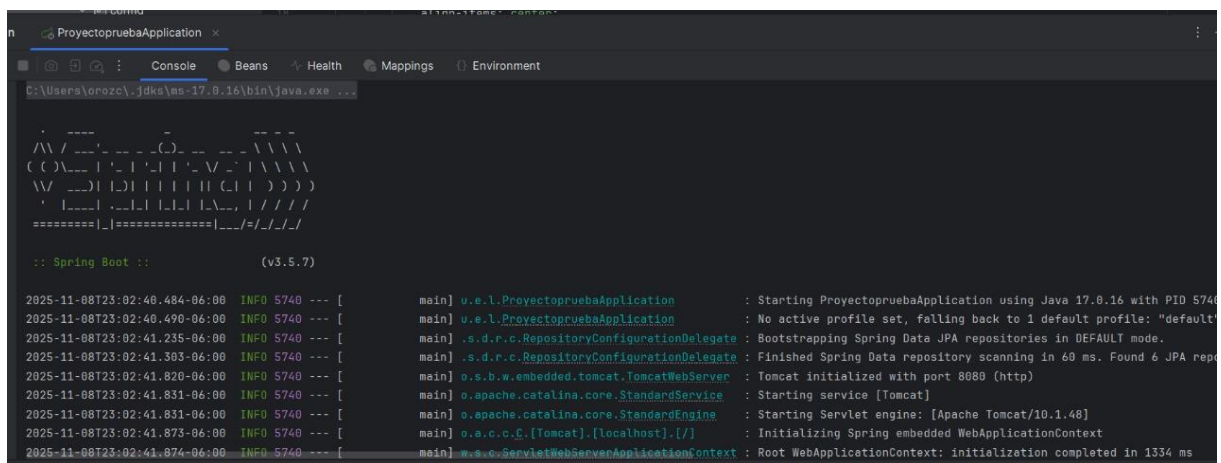
#	1	2 direccion	3 nombre	4 telefono
1	1	Zona 18, Los Olivos	Arleth Orozco	56316013
2	2	Zona 6	Maycol Cabrera	32522645
3	3	Zona 4	Mario Perez	55201024

En las propiedades para la conexión del código a base de datos nos solicita nuestro usuario y contraseña, así mismo el puerto utilizado.



```
1  # ? Puerto del servidor
2  server.port=8080
3
4  # ? Configuración de base de datos
5  spring.datasource.url=jdbc:mariadb://localhost:3306/logistica
6  spring.datasource.username=root
7  spring.datasource.password=12345
8
9  # ? Configuración JPA / Hibernate
10 spring.jpa.hibernate.ddl-auto=update
11 spring.jpa.show-sql=true
12 spring.jpa.properties.hibernate.format_sql=true
13
14 # ? Configuración JWT
15 jwt.secret=MiclaveSuperSecretaDeMasDe32Caracteres123456
16 jwt.expiration=3600000
17
```

Ya con el correcto funcionamiento procedimos a la base de datos



```
2025-11-08T23:02:40.484-06:00 INFO 5740 --- [main] u.e.l.ProyectopruebaApplication : Starting ProyectopruebaApplication using Java 17.0.16 with PID 5740
2025-11-08T23:02:40.490-06:00 INFO 5740 --- [main] u.e.l.ProyectopruebaApplication : No active profile set, falling back to 1 default profile: "default"
2025-11-08T23:02:41.235-06:00 INFO 5740 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2025-11-08T23:02:41.393-06:00 INFO 5740 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 60 ms. Found 6 JPA repositories
2025-11-08T23:02:41.820-06:00 INFO 5740 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2025-11-08T23:02:41.831-06:00 INFO 5740 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-11-08T23:02:41.831-06:00 INFO 5740 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.48]
2025-11-08T23:02:41.873-06:00 INFO 5740 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2025-11-08T23:02:41.874-06:00 INFO 5740 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1334 ms
```

Luego de realizar la conexión en si y que el codigo corriera de manera correcta, pasamos a nuestra base de datos para poder crear las diversas tablas que conforman el proyecto.

The screenshot shows a database management interface. On the left, a tree view displays the database structure under 'local'. The 'logistica' database is expanded, showing tables: clientes (16.0 KiB), entregas (48.0 KiB), ordenes_envio (32.0 KiB), roles (32.0 KiB), transportistas (16.0 KiB), usuarios (32.0 KiB), and usuario_roles (32.0 KiB). The 'entregas' table is selected. On the right, a table view shows the data for 'logistica.entregas', which contains 2 rows. The table has 5 columns: #, 1 id, 2 estado, 3 fecha_entrega, 4 orden_envio_id, and 5 transportista_id.

#	1 id	2 estado	3 fecha_entrega	4 orden_envio_id	5 transportista_id
1	1	Tardía	2025-10-31	1	1
2	2	A tiempo	2025-12-25	2	2

The screenshot shows the same database management interface. The 'ordenes_envio' table is now selected in the tree view. The table view on the right shows the data for 'logistica.ordenes_envio', which contains 3 rows. The table has 5 columns: #, 1 id, 2 direccion_destino, 3 estado, 4 fecha_envio, and 5 cliente_id.

#	1 id	2 direccion_destino	3 estado	4 fecha_envio	5 cliente_id
1	1	Zona 17, Portales	Pendiente	2025-10-25	1
2	2	Zona 17, Portales	Enviado	2025-10-23	2
3	3	Zona 10, Acropolis	Entregado	2025-10-31	3

The screenshot shows the database management interface. The 'roles' table is now selected in the tree view. The table view on the right shows the data for 'logistica.roles', which contains 2 rows. The table has 2 columns: #, 1 id, and 2 nombre.

#	1 id	2 nombre
1	1	ROLE_ADMIN
2	2	ROLE_USER

Filtro de bases de datos Filtro de tablas local Base de datos: logistica Tabla: transportistas

logistica.transportistas: 2 filas en total (exacto)

#	1 id	2 nombre	3 telefono	4 vehiculo
1	1	Alex Ramirez	55201512	Camion JAC
2	2	Karla Morales	85451520	Motocicleta

local

- information_schema
- logistica 208.0 KiB
 - clientes 16.0 KiB
 - entregas 48.0 KiB
 - ordenes_envio 32.0 KiB
 - roles 32.0 KiB
 - transportistas 16.0 KiB
 - usuarios 32.0 KiB
 - usuario_roles 32.0 KiB
- mysql
- performance_schema

Filtro de bases de datos Filtro de tablas local Base de datos: logistica Tabla: usuarios Datos Consulta*

logistica.usuarios: 1 filas en total (exacto)

#	1 id	2 a...	3 nombre	4 password	5 username
1	1	1	Administrador General	\$2a\$10\$hAqrV0yJDCfsl0ovDf4mq.C6YmgjthZ...	admin

local

- information_schema
- logistica 208.0 KiB
 - clientes 16.0 KiB
 - entregas 48.0 KiB
 - ordenes_envio 32.0 KiB
 - roles 32.0 KiB
 - transportistas 16.0 KiB
 - usuarios 32.0 KiB
 - usuario_roles 32.0 KiB
- mysql
- performance_schema
- sys

Filtro de bases de datos Filtro de tablas local Base de datos: logistica Tabla: usuario_roles Datos

logistica.usuario_roles: 1 filas en total (exacto)

#	1 usuario_id	2 rol_id
1	1	1

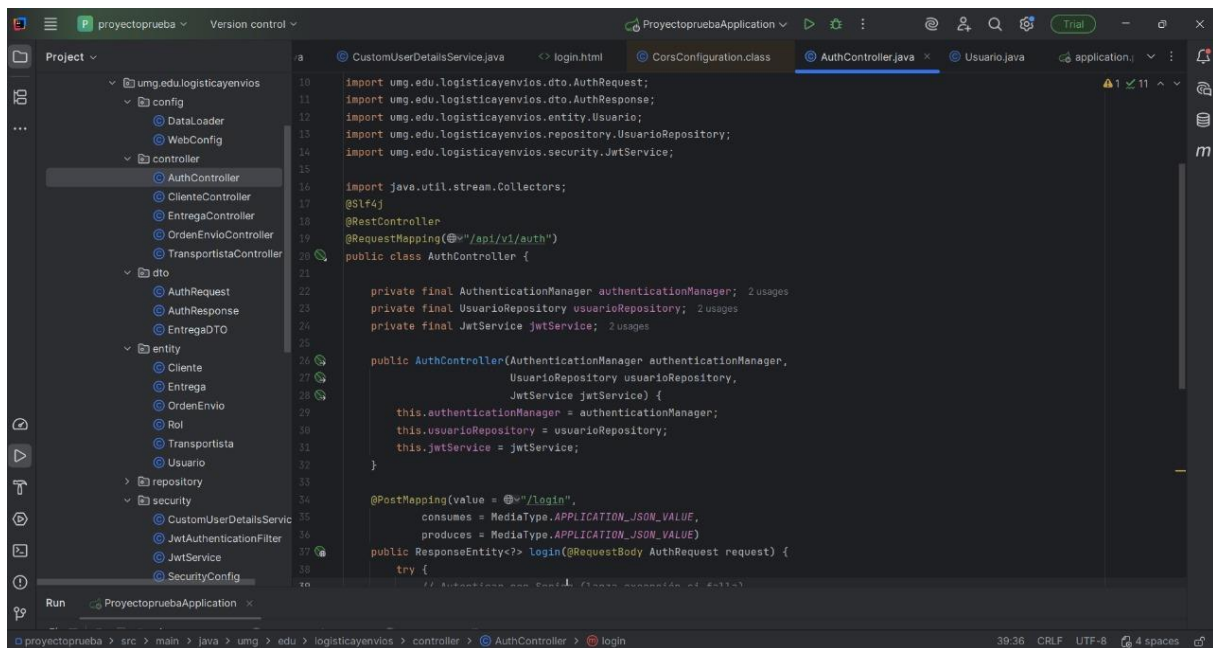
local

- information_schema
- logistica 208.0 KiB
 - clientes 16.0 KiB
 - entregas 48.0 KiB
 - ordenes_envio 32.0 KiB
 - roles 32.0 KiB
 - transportistas 16.0 KiB
 - usuarios 32.0 KiB
 - usuario_roles 32.0 KiB
- mysql
- performance_schema
- sys

Luego pasamos a configurar detalles en el código como lo es El CorsConfiguration.class que se utiliza para habilitar las solicitudes entre diferentes orígenes.

Como el proyecto usa JavaScript en el frontend y Spring Boot en el backend, el navegador normalmente bloquearía la comunicación por razones de seguridad.

Esta configuración permite autorizar esas solicitudes, indicando qué direcciones pueden acceder, qué métodos se pueden usar y qué encabezados.



The screenshot shows an IDE window for a project named 'proyectoprueba'. The file explorer on the left shows the project structure, including a 'controller' package with 'AuthController.java'. The main editor displays the 'CorsConfiguration.class' file, which is currently empty. The 'Run' button at the bottom left is highlighted, and the status bar at the bottom shows the file path: 'proyectoprueba > src > main > java > umg > edu > logisticyenvios > controller > AuthController > login'.

```
import umg.edu.logisticyenvios.dto.AuthRequest;
import umg.edu.logisticyenvios.dto.AuthResponse;
import umg.edu.logisticyenvios.entity.Usuario;
import umg.edu.logisticyenvios.repository.UsuarioRepository;
import umg.edu.logisticyenvios.security.JwtService;

import java.util.stream.Collectors;

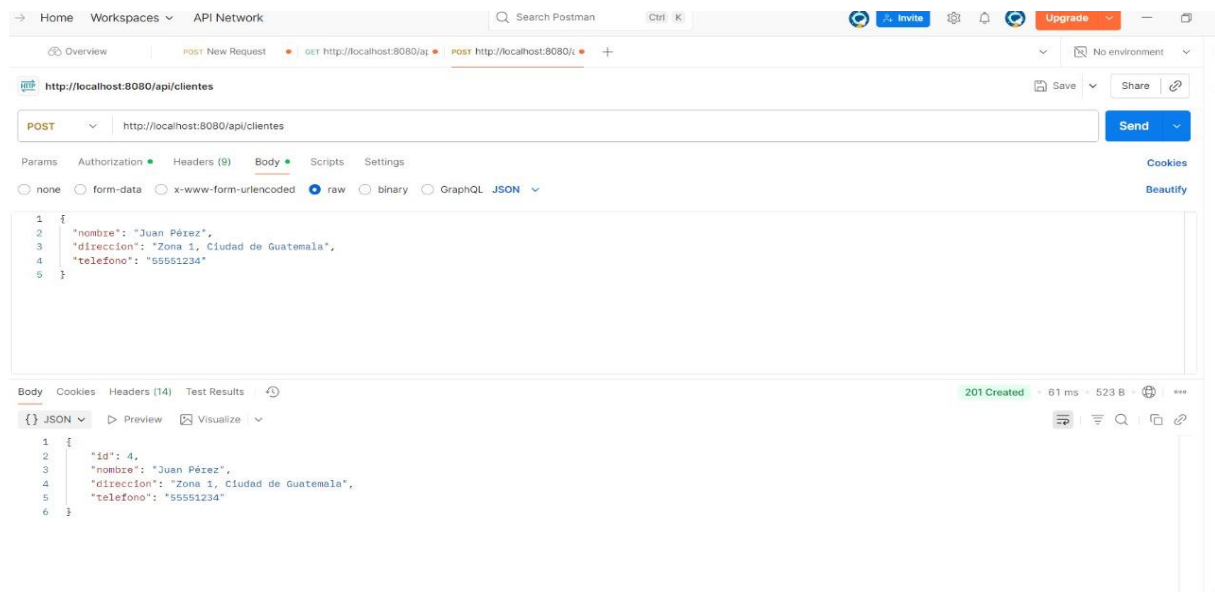
@Slf4j
@RestController
@RequestMapping("/api/v1/auth")
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final UsuarioRepository usuarioRepository;
    private final JwtService jwtService;

    public AuthController(AuthenticationManager authenticationManager,
        UsuarioRepository usuarioRepository,
        JwtService jwtService) {
        this.authenticationManager = authenticationManager;
        this.usuarioRepository = usuarioRepository;
        this.jwtService = jwtService;
    }

    @PostMapping(value = "/login",
        consumes = MediaType.APPLICATION_JSON_VALUE,
        produces = MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity<> login(@RequestBody AuthRequest request) {
        try {
            // Authentication and Token Generation Logic
        }
    }
}
```

Sabemos que las APIs son como *puertas* por donde el frontend y otros sistemas pueden pedir información u ordenar acciones al backend. Por lo cual procedemos a crear las diversas colecciones, requests, los scrips y las variables en si.



Overview | POST New Request | GET http://localhost:8080/api/ | POST http://localhost:8080/api/ | + | No environment

http://localhost:8080/api/clientes

POST | http://localhost:8080/api/clientes

Params | Authorization | Headers (9) | Body | Scripts | Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON

```
1 {
2   "nombre": "Max Smith",
3   "direccion": "Zona 2, El Zapote",
4   "telefono": "88541110"
5 }
```

Body | Cookies | Headers (14) | Test Results | 201 Created · 19 ms · 511 B

```
{
  "id": 5,
  "nombre": "Max Smith",
  "direccion": "Zona 2, El Zapote",
  "telefono": "88541110"
}
```

Overview | POST New Request | GET http://localhost:8080/api/ | POST http://localhost:8080/api/ | GET http://localhost:8080/api/ | + | No environment

http://localhost:8080/api/entregas

GET | http://localhost:8080/api/entregas

Params | Authorization | Headers (7) | Body | Scripts | Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON

1. Ctrl+Alt+F to Ask AI

Body | Cookies | Headers (14) | Test Results | 200 OK · 37 ms · 1.11 KB

```
{
  "id": 1,
  "estado": "Tardía",
  "fechaEntrega": "2025-10-31",
  "ordenEnvio": {
    "id": 1,
    "direccionDestino": "Zona 17, Portales",
    "fechaEnvio": "2025-10-26",
    "estado": "Pendiente",
    "cliente": {
      "id": 1,
      "nombre": "Arleth Orozco"
    }
  }
}
```

Overview | POST New Request | GET http://localhost:8080/api/ | POST http://localhost:8080/api/ | GET http://localhost:8080/api/ | POST http://localhost:8080/api/ | PUT http://localhost:8080/api/ | + | No environment

http://localhost:8080/api/clientes/4

PUT | http://localhost:8080/api/clientes/4

Params | Authorization | Headers (9) | Body | Scripts | Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON

```
1 {
2   "nombre": "Max Steel",
3   "direccion": "Zona 10, Ciudad de Guatemala",
4   "telefono": "55555678"
5 }
6
```

Body | Cookies | Headers (14) | Test Results | 200 OK · 27 ms · 517 B

```
{
  "id": 4,
  "nombre": "Max Steel",
  "direccion": "Zona 10, Ciudad de Guatemala",
  "telefono": "55555678"
}
```

My Collection / **Get data**

GET http://localhost:8080/api/ordenes Send

Params Authorization Headers (7) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

Schema Beautify

1 Ctrl+Alt+P to Ask AI

Body Cookies Headers (14) Test Results (1/1) 200 OK - 14 ms - 1.1 KB Save Response

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "direccionDestino": "Zona 17, Portales",
4   "fechaEnvio": "2025-10-26",
5   "estado": "Pendiente",
6   "cliente": {
7     "id": 1,
8     "nombre": "Arleth Orozco ",
9     "direccion": "Zona 18, Los Olivos",
10    "telefono": "56316013"
11  },
12 }
```

Find and replace Console Runner Start Proxy Cookies Vault Trash

My Collection / **Post data**

POST http://localhost:8080/api/transportistas Send

Params Authorization Headers (10) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

Schema Beautify

```
1 {
2   "nombre": "Juan Pérez",
3   "telefono": "5551-2345",
4   "vehículo": "Camioneta"
5 }
6
7
8
```

Body Cookies Headers (14) Test Results (2/2) 200 OK - 19 ms - 516 B Save Response

JSON Preview Visualize

```
1 {
2   "id": 3,
3   "nombre": "Juan Pérez",
4   "telefono": "5551-2345",
5   "vehículo": "Camioneta",
6   "entregas": null
7 }
```

Find and replace Console Runner Start Proxy Cookies Vault Trash

My Collection / **Borrar**

DELETE http://localhost:8080/api/transportistas/3 Send

Params Authorization Headers (8) Body Scripts Settings

Auth Type

Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

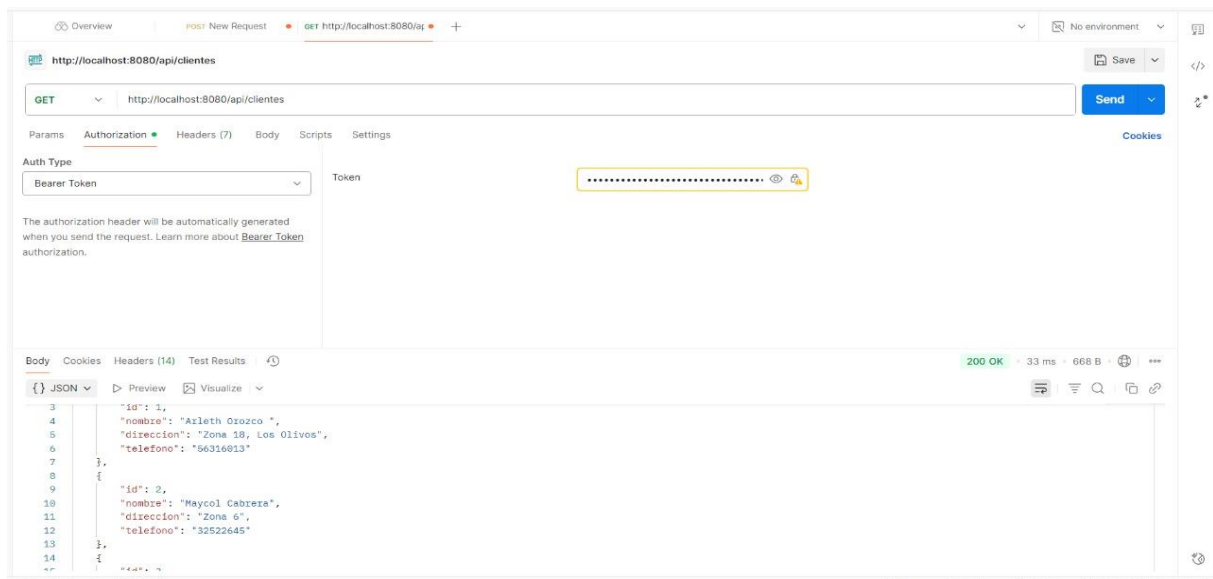
Token

.....

Body Cookies Headers (12) Test Results 204 No Content - 299 ms - 371 B Save Response

Raw Preview Visualize

1



Luego de tener las colecciones en si procedimos a la conexión con la web. Este es la relación que tiene con el login y el index donde la pagina web funcional, también es en donde se toma la relación del proyecto con la web.

```

+-----+      +-----+
| login.html |    | AuthController |
|-----| POST |-----|
| - username |----->| + login(username, |
| - password |    | password): Token |
| + login()   |    +-----+
+-----+      |
                | 200 OK
                v
                +-----+
                | JWT Token |
                +-----+
                |
                v
+-----+      +-----+
| index.html / | GET | ClienteController |
| clientes.html |----->| OrdenController |

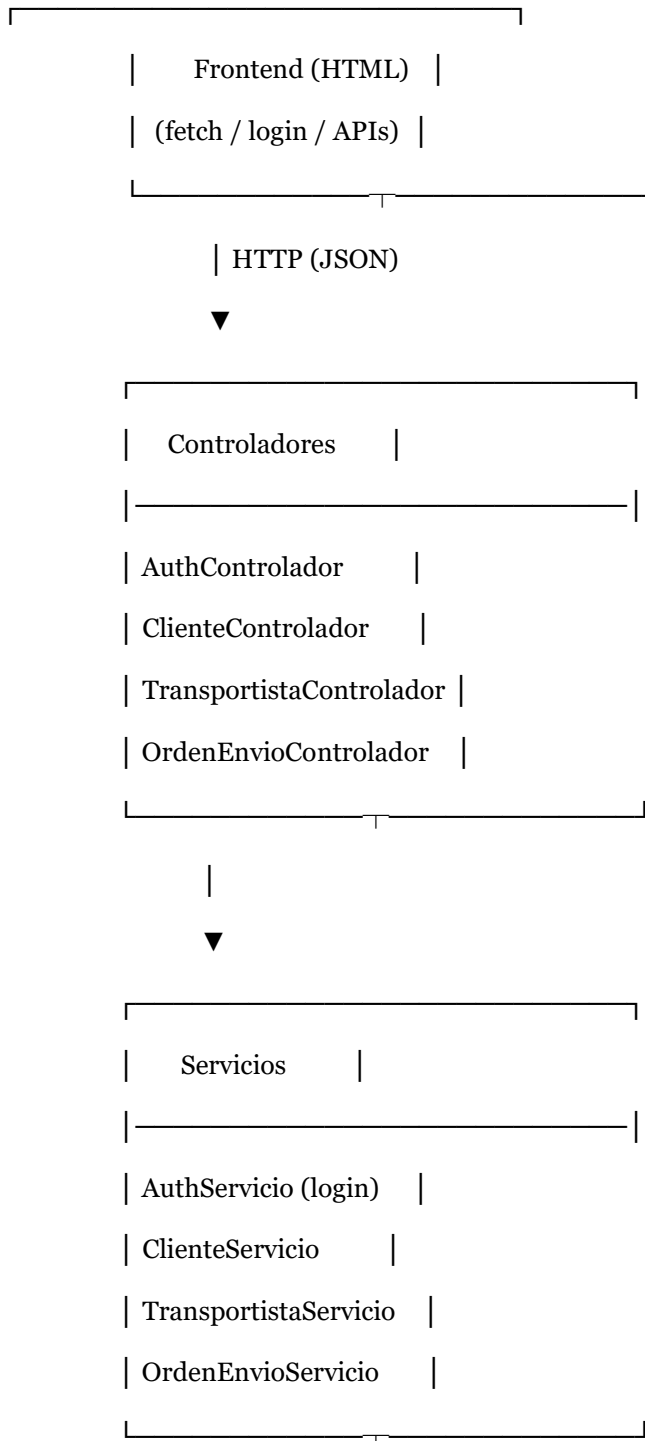
```

```

|-----| Bearer | TransportistaController |
| - token JWT | token | EntregaController |
| + fetchData(url) |----->| ... |
+-----+ +-----+

```

Asi mismo esta es la estructura base del proyecto



|



Repositorios
UsuarioRepositorio
RolRepositorio
ClienteRepositorio
TransportistaRepositorio
OrdenEnvioRepositorio
RutaRepositorio

| JPA / Hibernate

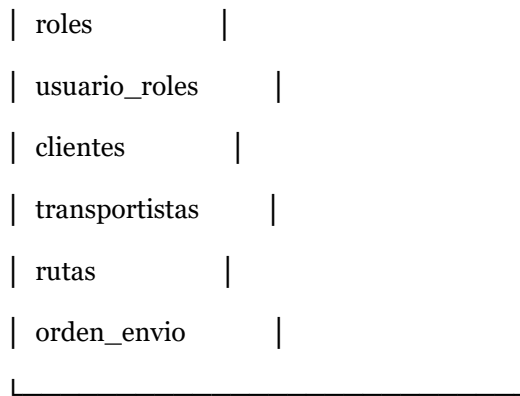


Entidades JPA
Usuario 1—* Rol (M:N)
Cliente 1—* OrdenEnvio
Transportista 1—* OrdenEnvio
Ruta 1—* OrdenEnvio

|



MariaDB
logistica_db
Tablas:
usuarios



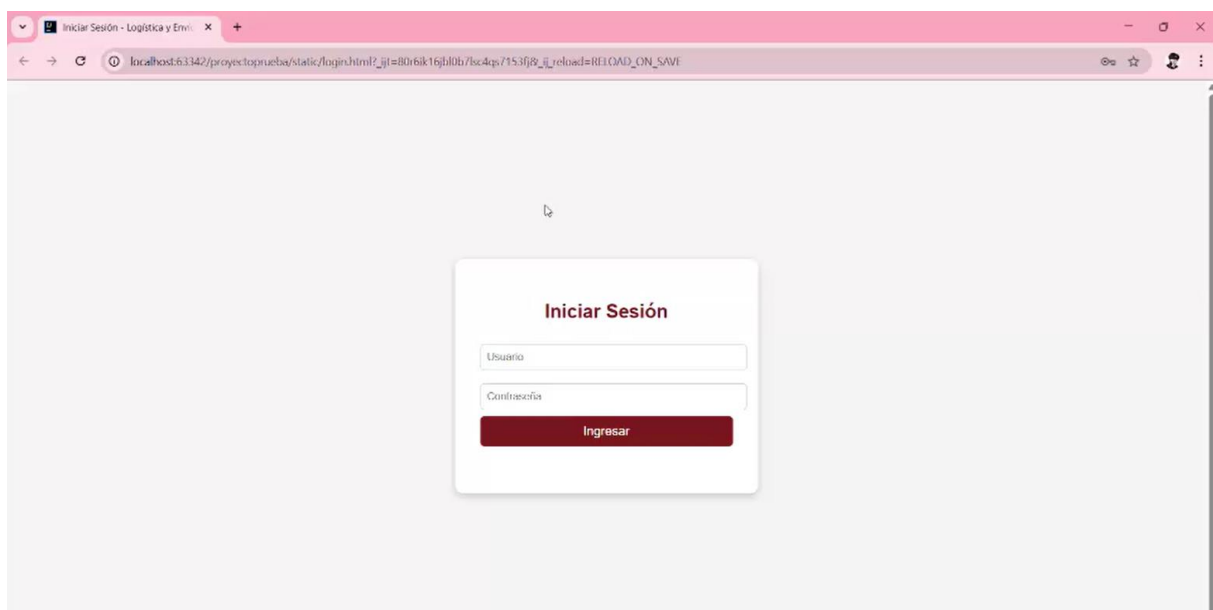
También compartimos un diagrama de flujo en el cual presentamos la arquitectura por capas del sistema.

Primero tenemos los Controllers, que reciben las solicitudes del usuario. Luego los Services, que contienen la lógica del negocio. Después los Repositories, que se encargan de comunicarse con la base de datos. Finalmente están las Entidades, que representan las tablas en MariaDB, donde se almacena la información.

Este diseño permite que el sistema sea organizado, fácil de mantener y escalable.

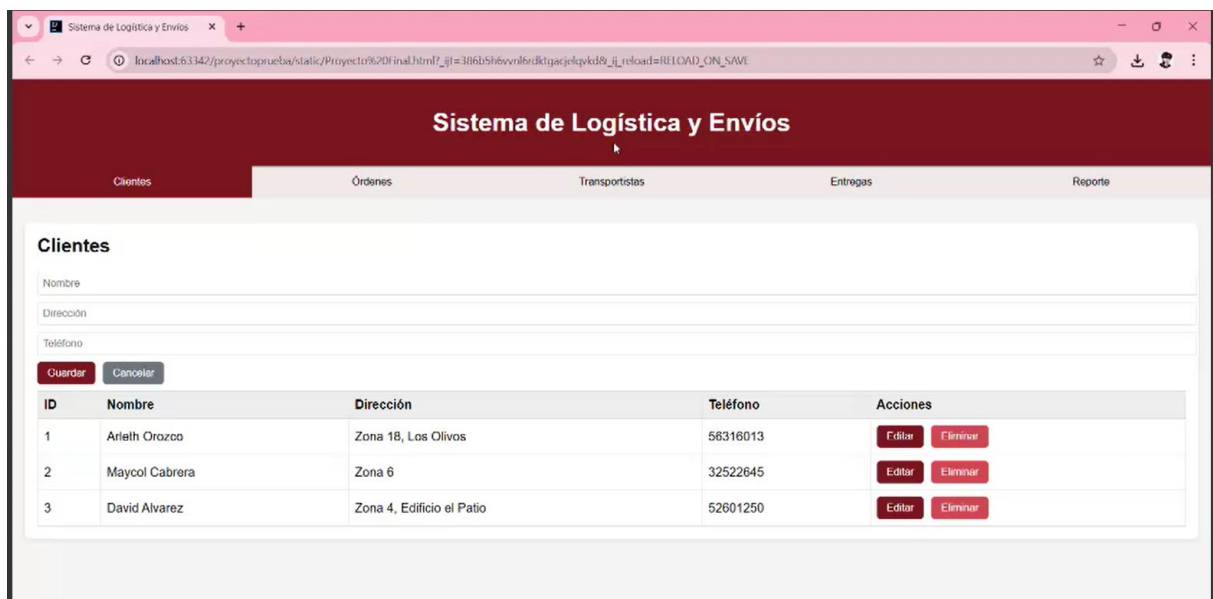
Luego de tener todo el proyecto configurado totalmente, lo que es el Java, spring boot, base de datos y postman, nos vamos directamente al funcionamiento en la web en la cual ya se presenta de manera profesional compartiendo un diseño moderno y llamativo, procedemos con la documentación de su correcto funcionamiento.

Al inicio solicita el Login en donde debemos iniciar con usuario y contraseña



The screenshot shows a web browser window with the title 'Iniciar Sesión - Logística y Envíos'. The address bar shows a localhost URL. The main content area features a centered login form with the title 'Iniciar Sesión'. The form contains two input fields labeled 'Usuario' and 'Contraseña', and a red button labeled 'Ingresar'.

Al iniciar sesión procedemos con a ingresar nuestros datos como clientes



The screenshot shows the 'Sistema de Logística y Envíos' dashboard. The top navigation bar is dark red with the system name. Below it is a horizontal menu with tabs: 'Clientes', 'Órdenes', 'Transportistas', 'Entregas', and 'Reporte'. The 'Clientes' tab is active, displaying a form to add a new client and a table of existing clients.

Clientes

Form fields: Nombre, Dirección, Teléfono. Buttons: Guardar, Cancelar.

ID	Nombre	Dirección	Teléfono	Acciones
1	Arielh Orozco	Zona 18, Los Olivos	56316013	<button>Editar</button> <button>Eliminar</button>
2	Maycol Cabrera	Zona 6	32522645	<button>Editar</button> <button>Eliminar</button>
3	David Alvarez	Zona 4, Edificio el Patio	52601250	<button>Editar</button> <button>Eliminar</button>

Sistema de Logística y Envíos

Cientes

Órdenes de Envío

Transportistas

Entregas

Órdenes de Envío

Arieth Orozco

Dirección de destino

mm/dd/yyyy

Pendiente

Guardar

Cancelar

ID	Cliente	Destino	Fecha	Estado	Acciones
1	Arieth Orozco	Zona 5, Calzada la Paz	2025-02-11	Pendiente	Editar Eliminar
2	David Alvarez	Zona 17, Portales	2025-03-12	Pendiente	Editar Eliminar

Sistema de Logística y Envíos

Cientes

Órdenes de Envío

Transportistas

Entregas

Transportistas

Nombre

Teléfono

Vehículo

Guardar

Cancelar

ID	Nombre	Teléfono	Vehículo	Acciones
1	Alex Ramirez	58261502	Pick Up	Editar Eliminar
2	Manuel Medrano	52631485	Camion	Editar Eliminar

Asi mismo adjuntamos un video por medio de enlace a drive, en el cual se muestra y da a conocer el procedimiento y funcionamiento del proyecto.

<https://drive.google.com/file/d/1f558mgQgfc3um7cHCHl-Zeq6envqjsSh/view?usp=sharing>